Business 41204

Machine Learning

Introduction to Unsupervised Learning

Outline:

1. Introduction
2. Clustering
3. Introduction to Encoding: Dimension Reduction and PCA

1. Introduction

Unsupervised Learning

Unsupervised learning fundamentally about learning “representations”

- “low-dimensional” summary that captures important features of data
  - E.g., sample mean = low-dimensional summary of a column of numbers
- Can think about as “fancy” descriptive statistics

Canonical examples:

- Clustering = grouping observations or variables
  - Anomaly detection (really just clustering focused on finding things that don’t fit well into groups)

- Dimension reduction = finding a small set of factors that capture most of the variation in a large set of variables
  - Matrix completion (“Netflix problem” – really just dimension reduction)

Unsupervised learning often used as input into further downstream task

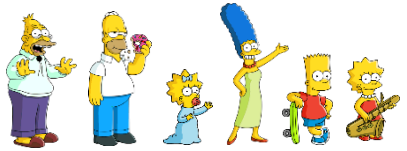
Challenge

Key challenge of USL: often subjective

- No specific, pre-specified target in the data (as in supervised learning)
- No explicit rewards from environment (as in reinforcement learning – LATER)
- Whether representation is useful is often in the eye of the beholder

Example: Data = 

Representation 1

Group 1 

Group 2 

This representation would actually be very hard to learn from the provided data…

Representation 2

Group 1 

Group 2 

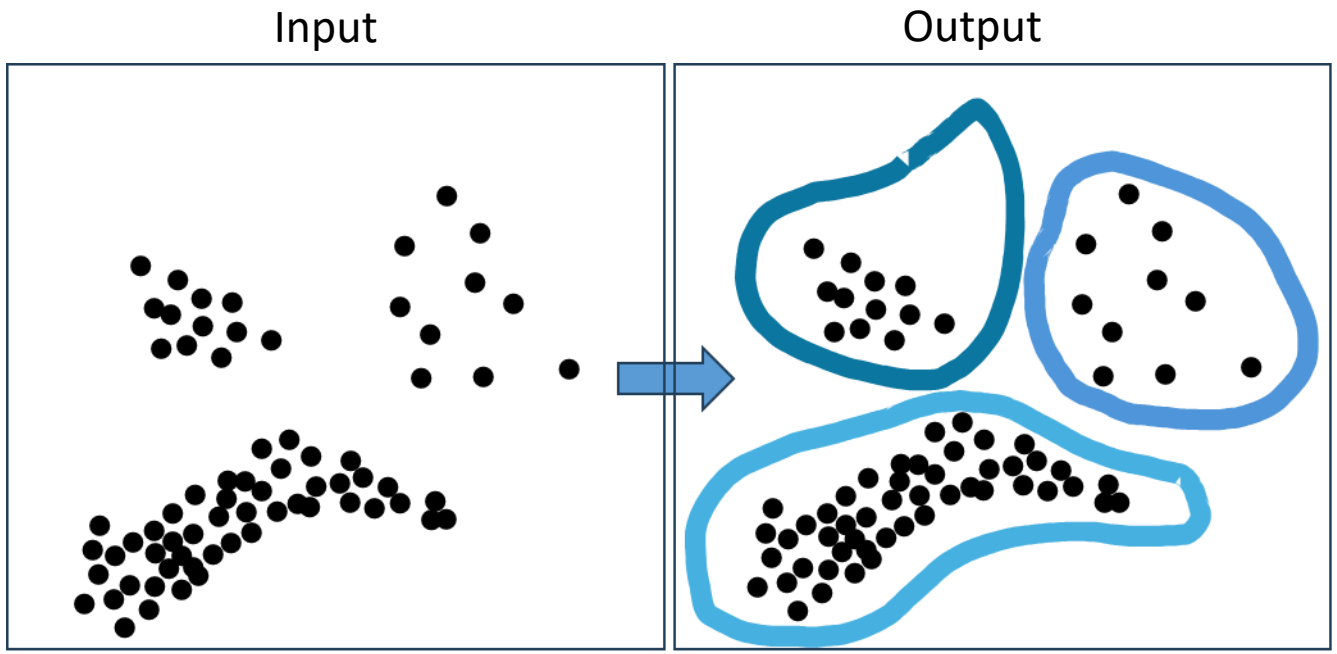
Both representations are "correct" and provide "low-dimensional" summaries of the data. Neither is obviously "better" than the other.

2. Clustering

Clustering

The main idea behind *clustering* is to find groups of observations that are similar to each other

- but not so similar to other observations
- essentially any method for automatically partitioning data



Exploratory tool:

- way to find interesting patterns in data

Can be used to perform data reduction:

- E.g., “treat” everyone in a group the same way.

Clustering

Goal: Automatically and reliably group “similar” data together into “clusters”

- Provides a way to talk about complex data
- Allows different analytics to be applied to different groups
- If data are similar in some ways, they often share similarities in others
  - Essentially the same idea underlies supervised learning when we have a well-specified, observed target

Operationalizing requires

- A definition of similarity
  - Typically, we use a “distance” (d) where more distant observations are deemed less similar
- A clustering algorithm that works with d

Applications:

- Recommender systems
  - “People who like *X* also like *Y*”
  - “Customers with similar browsing histories also looked at …”
- Market segmentation
- Classification
  - Taxonomy is just clustering

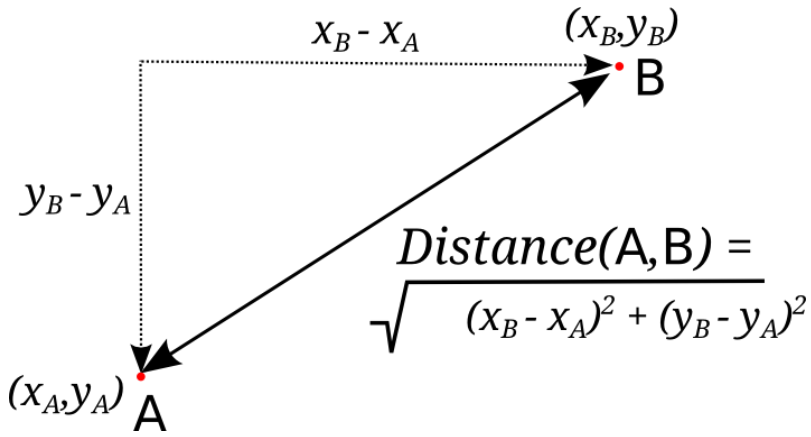
Distance as a Measure of Similarity

	Age	Years at current address	Residential Status (1=owner, 2=renter)
Person A	23	4	2
Person B	40	10	1

How to compute similarity between person A and person B?

- Many possible choices
- Units are important – usually people standardize data before clustering

Euclidean ("triangle") distance is the most common choice



Euclidean Distance

		x_1	x_2	x_3	…	x_d
Observation i	x_i	x_{i1}	x_{i2}	x_{i3}	…	x_{id}
Observation j	x_j	x_{j1}	x_{j2}	x_{j3}	…	x_{jd}

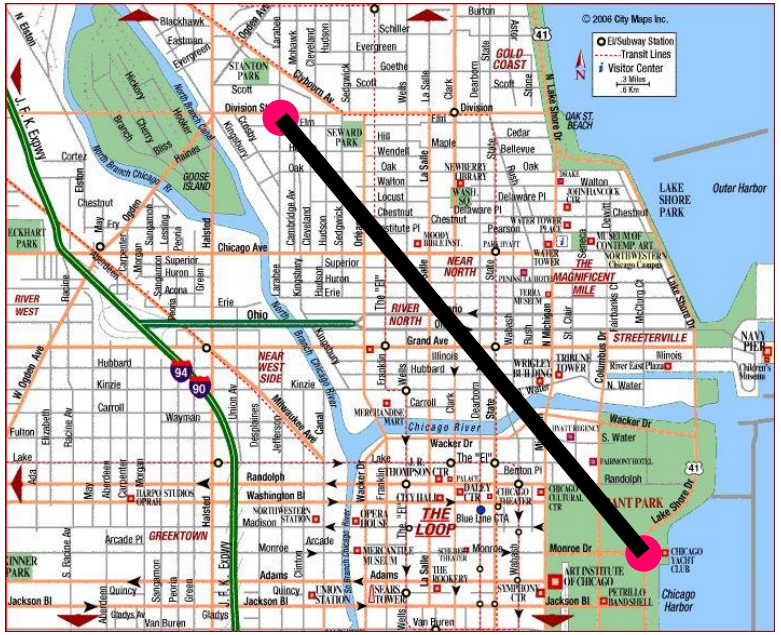
$$dist(x_i, x_j) = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \cdots + (x_{id} - x_{jd})^2}$$

In our toy example:

- Person A: Age = 23, Years at current address = 4, Residential Status = 2
- Person B: Age = 40, Years at current address = 10, Residential Status = 1

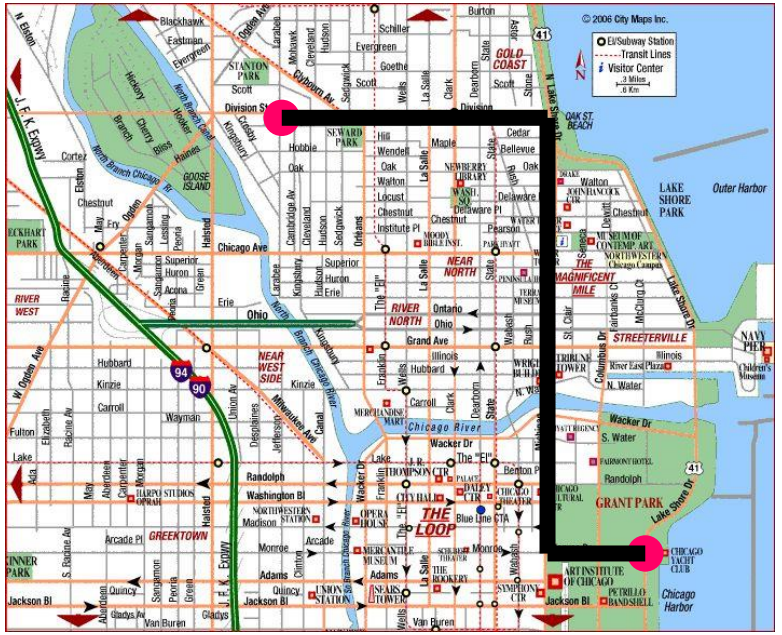
$$dist(person_A, person_B) = \sqrt{(23-40)^2 + (4-10)^2 + (2-1)^2} = \sqrt{86} = 9.274$$

Common Distances



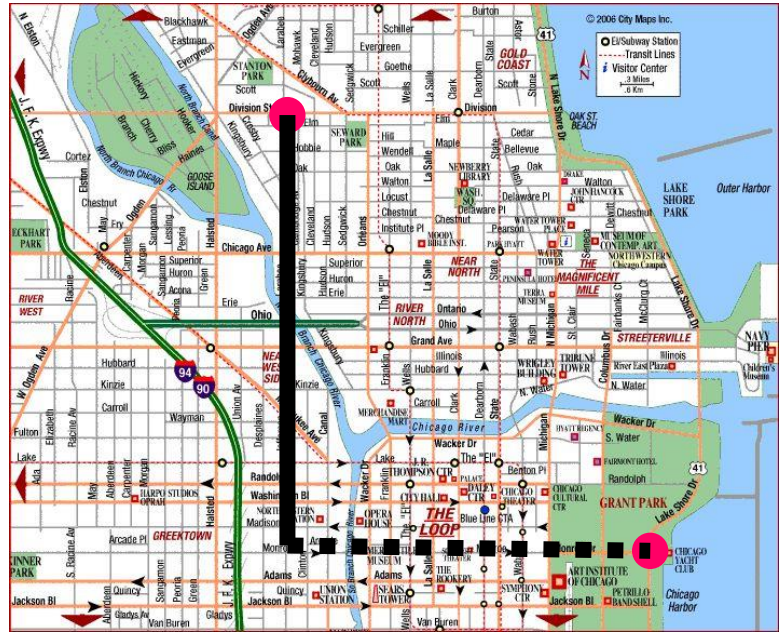
Euclidean distance

- "as the crow flies"
- ℓ_2 distance



Manhattan distance

- "driving on a grid"
- ℓ_1 distance



Sup distance

- "only worry about longest"
- ℓ_∞ distance

- All are different – produce different clusters
- Can be mixed
- Units are probably more important

K-Means Clustering

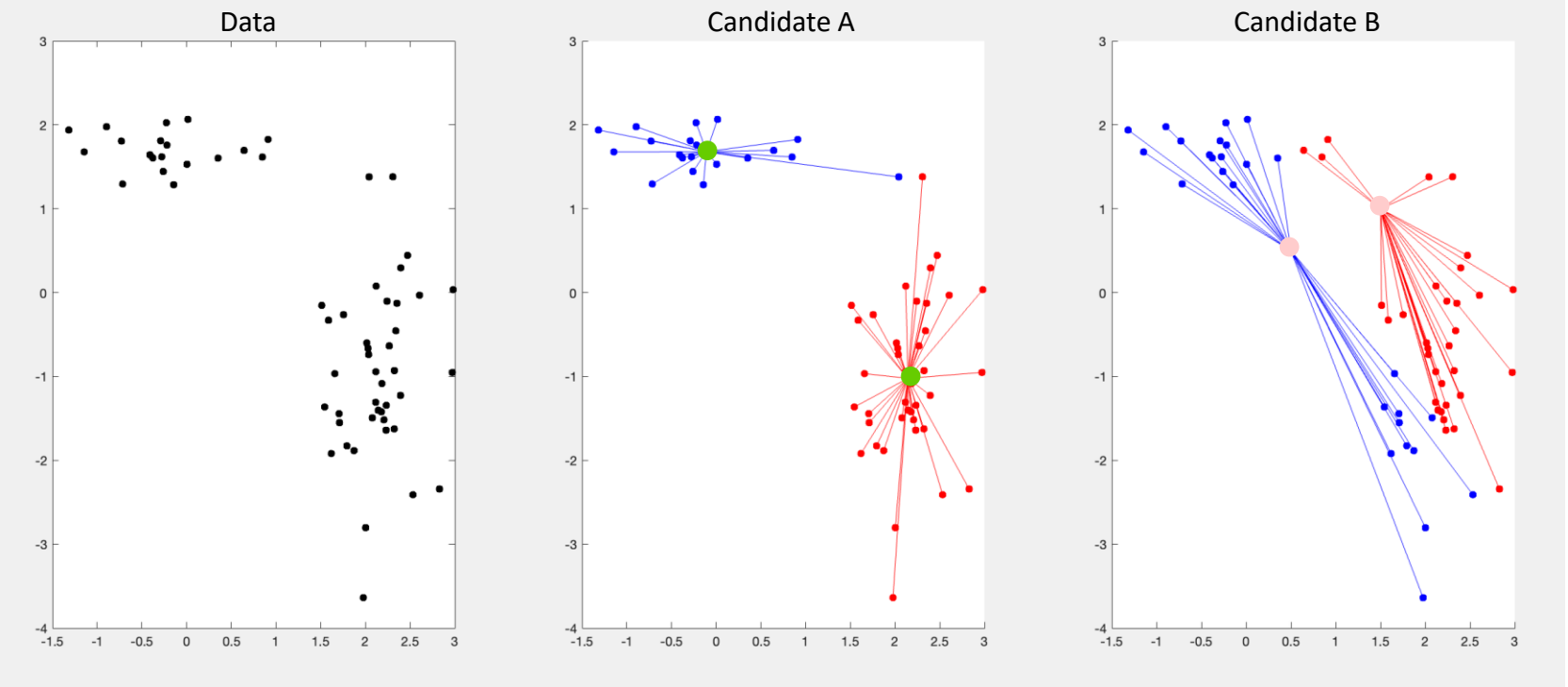
Once we've decided how to define similarity, we want to use this measure to group similar observations together.

Most common method used to take similarities and provide groups (clusters) is k-means clustering

- Find K cluster centers $k_1, k_2, \ldots, k_K$ and K groups (clusters) of observations $C_1, C_2, \ldots, C_K$ to minimize

$$\sum_{l=1}^{K} \sum_{\substack{observation\ i \\ is\ in\ cluster\ C_l}} dist(x_i, k_l)$$

Visualizing the K-means Problem



- Cluster centers for candidate grouping A
- Cluster centers for candidate grouping B

Candidate A: Sum of distances to cluster centers = 12.5419 + 40.5925 = 53.1344

Candidate B: Sum of distances to cluster centers = 50.1245 + 57.3321 = 107.4566

Prefer Candidate A to Candidate B because 53 < 107

Code for shopping example

Example: Market Segmentation

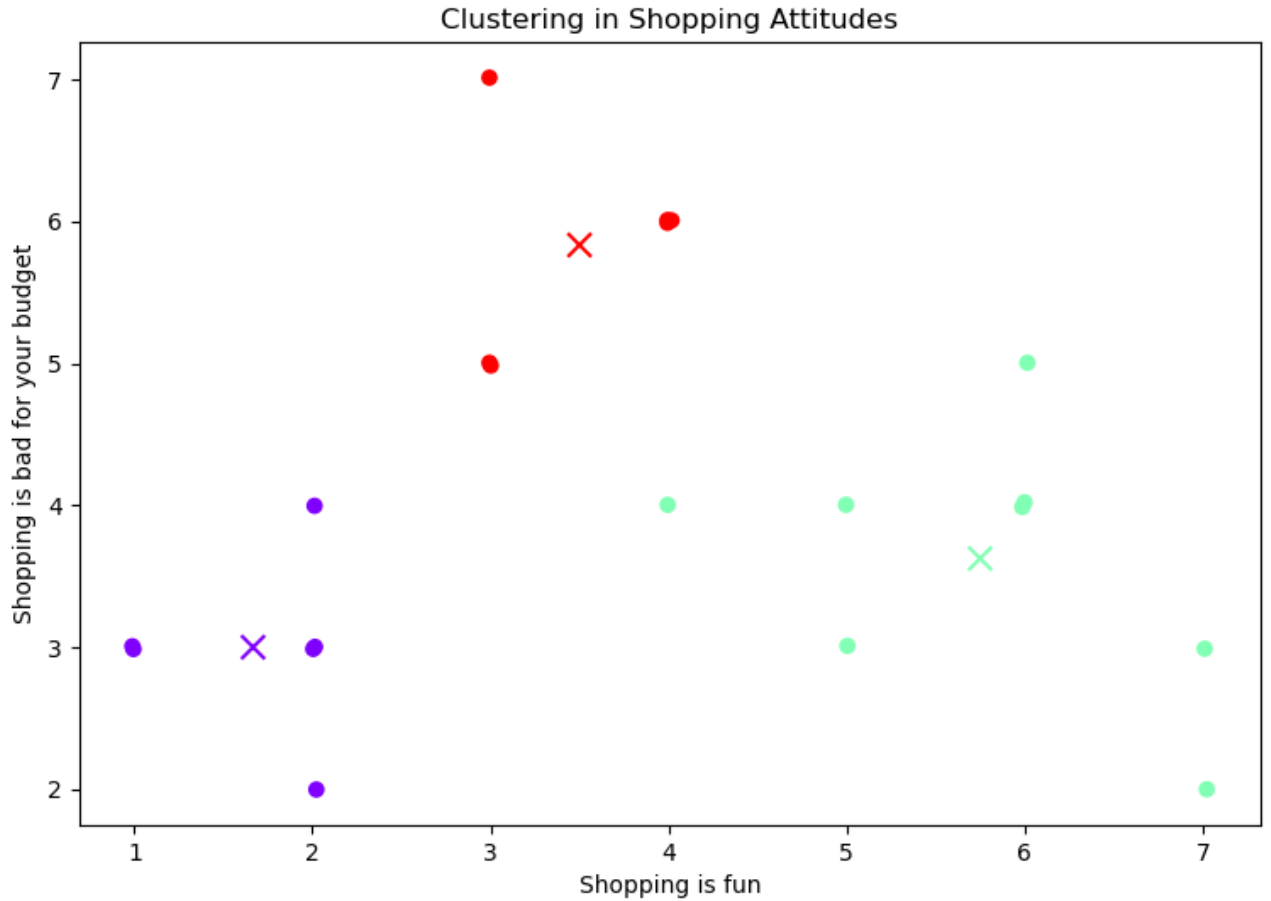
Suppose we wish to group customers on the basis of shopping attitudes

Have asked some customers to answer questions about their shopping attitudes:

- V1: Shopping is fun
- V2: Shopping is bad for your budget
- V3: I combine shopping with eating out
- V4: I try to get the best buys while shopping
- V5: I don't care about shopping
- V6: You can save a lot of money by comparing prices

Responses: 1 = strongly disagree, …, 7 = strongly agree

Shopping Attitude Clusters



Segments produced from using 3 clusters

For visualization, just looking at the first two variables

- Shopping is fun
- Shopping is bad for your budget

Shopping Attitude Cluster Centers

	Cluster		
	1	2	3
Shopping is fun	**1.67**	**5.75**	3.50
Shopping is bad for your budget	3.00	3.63	**5.83**
I combine shopping with eating out	**1.83**	**6.00**	3.33
I try to get the best buys while shopping	3.50	3.13	**6.00**
I don't care about shopping	**5.50**	**1.88**	3.50
You can save a lot of money by comparing prices	3.33	3.88	**6.00**

Dislike shopping

Like shopping

Price conscious

Responses: 1 = strongly disagree, 7 = strongly agree

If we had other characteristics, we could also see if there are other shared features within these constructed groups.

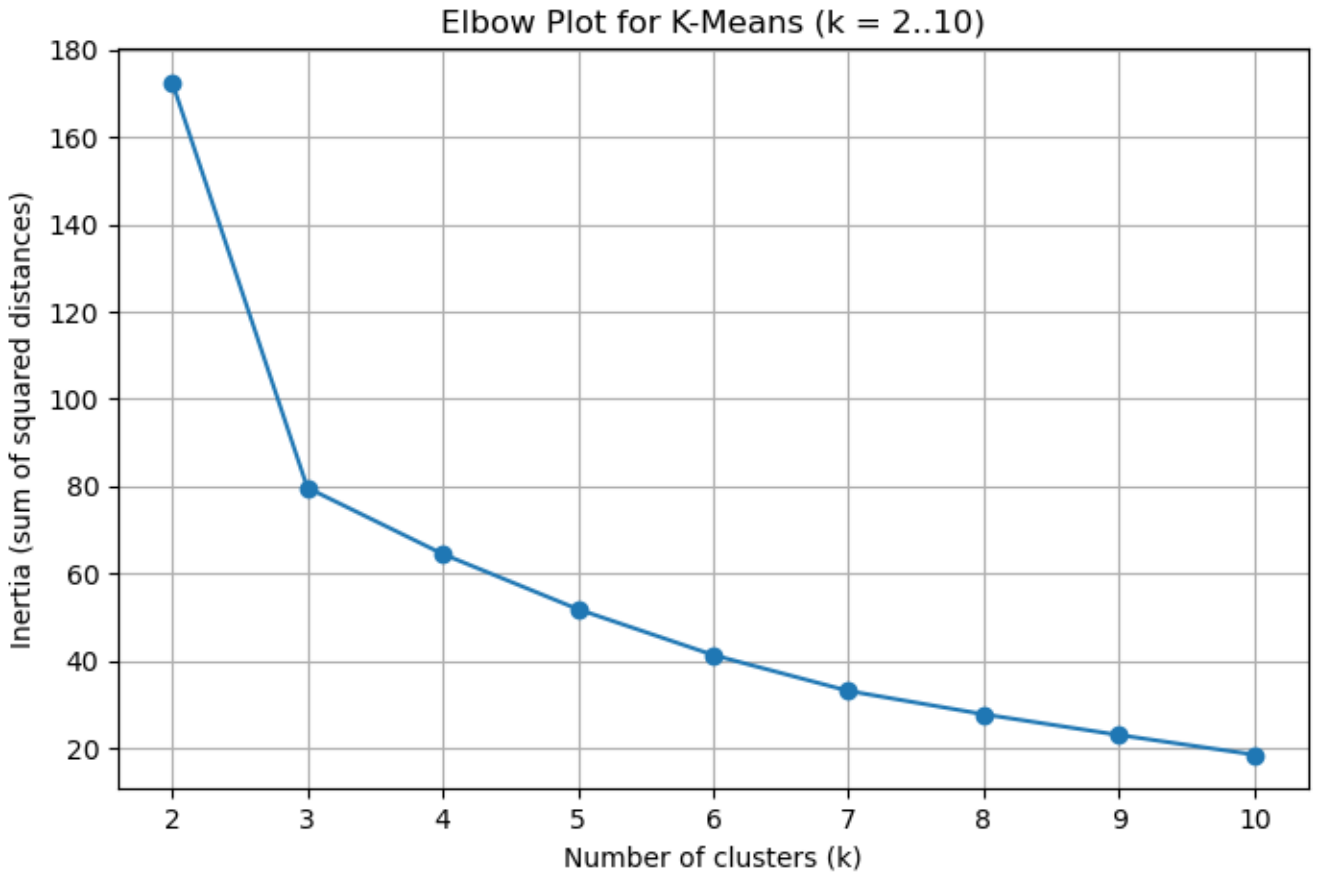
Choice of K

The number of clusters/groups is a tuning parameter

A common statistical heuristic is to choose based on the “elbow plot”

- Pick the point where diminishing returns seem to kick in – the “elbow”

I like choosing by looking at the resulting clusters and asking if they seem to capture something interesting



Code for whisky example

Example: Single Malt Scotch Whisky

Data with tasting notes on 109 Scotch whiskies

Tasting notes:

- **color** (14 values): yellow, very pale, pale, pale gold, gold, old gold, full gold, amber, etc.
- **nose** (12 values): aromatic, peaty, sweet, light, fresh, dry, grassy, etc.
- **body** (8 values): soft, medium, full, round, smooth, light, firm, oily
- **palate** (15 values): full, dry, sherry, big, fruity, grassy, smoky, salty, etc.
- **finish** (19 values): full, dry, warm, light, smooth, clean, fruity, grassy, etc.

Total of 68 characteristics.

- binary variables: whether Scotch has a particular characteristic or not.

Recommendation?

We just tried Ardbeg and really liked it. Can we find other Scotches similar to it?

Tasting notes for Ardbeg:

- Color: sherry
- Nose: peat, dry, sea
- Body: medium, full, light, firm
- Palate: sweet
- Finish: salt

How do we measure similarity?

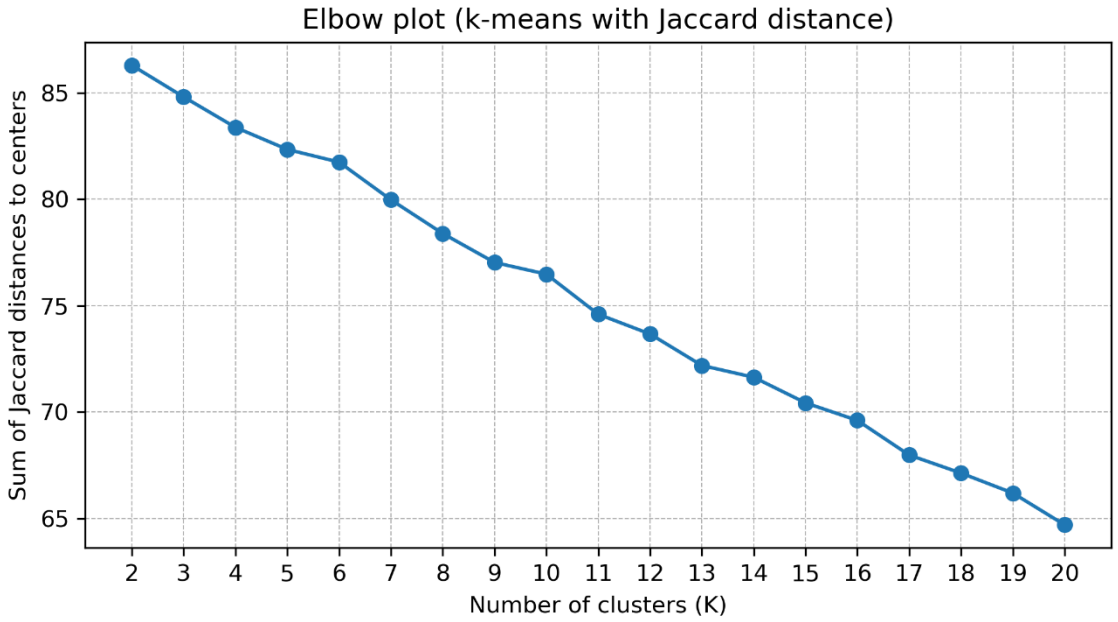
- Euclidean distance makes sense but feels a little off given the data
- Let's try Jaccard distance: $\mathrm{d}(W_A, W_B) = 1 - \frac{number\ of\ 1's\ common\ to\ A\ and\ B}{number\ of\ 1's\ across\ A\ and\ B}$
  - Focuses on "present" notes. Makes sense in a setting where shared absences are not particularly informative. (Most notes are absent in most whiskys.)

Ardbeg's Neighbors

Scotch	Jaccard distance	Description
Ardbeg	0	sherry; peat, dry, sea; medium, full, light, firm; sweet; salt
Bowmore	0.46	old gold; peat, dry, grass, sea; medium, light; sweet; salt, quick
Inchgower	0.65	gold; aroma, peat, sweet, dry, sea, rich; medium, smooth; sweet; salt; salt, long
Pulteney	0.65	amber; fresh, dry, sea; light, firm; smooth, sweet, spice, salt; warm, salt, long
Bunnahabhain	0.67	gold; fresh, sea; medium, light, firm; clean, fruit, sweet; full
Glenury Royal	0.67	bronze; peat, dry; medium; light, firm; dry, smoke, sweet; dry, fruit, smoke, sweet very long

K-Means

No obvious elbow – not atypical



Ardbeg's cluster mates:

5 clusters: Ardbeg, Benriach, Benrinnes, Bowmore, Convalmore, Craigellachie, Dailuaine, Glencadam, Glen Deveron, Glen Garioch, Glenlossie, Glen Mhor, Glen Ordie, Glenrothes, Glen Scotia, Imperial, Inchgower, Jura, Linkwood, Lochnagar, North Port, Oban, Pulteney, Springbank, Tobermory

10 clusters: Ardbeg, Bowmore, Brackla, Coleburn, Glenlivet, Oban, Port Ellen, Pulteney, Scapa

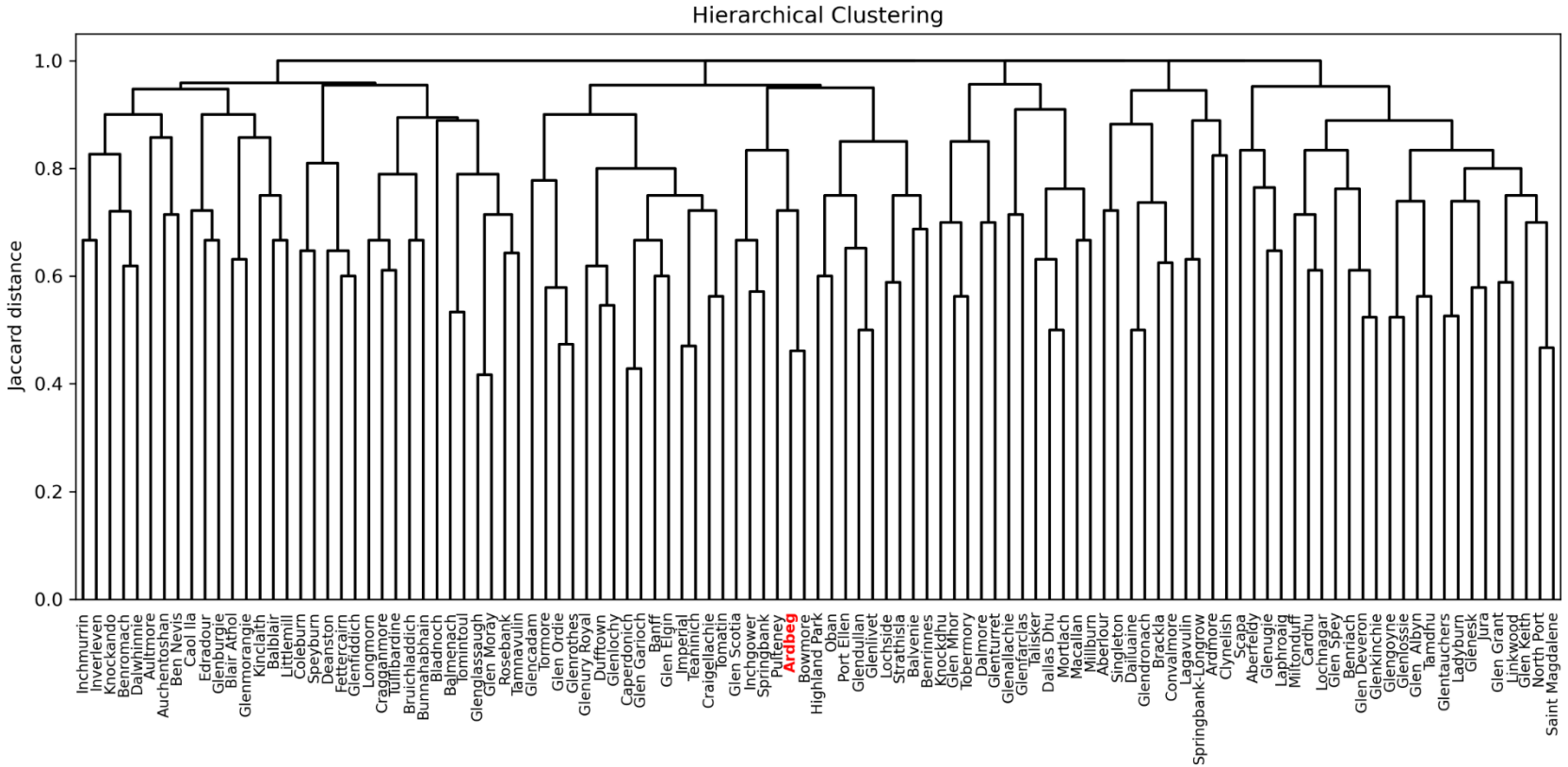
20 clusters: Ardbeg, Inchgower, Oban, Pulteney, Springbank

Which seems most useful?

Are these closest to Ardbeg?

Hierarchical Clustering

(Agglomerative) hierarchical clustering builds up clusters from the “bottom up” by grouping objects that are closest



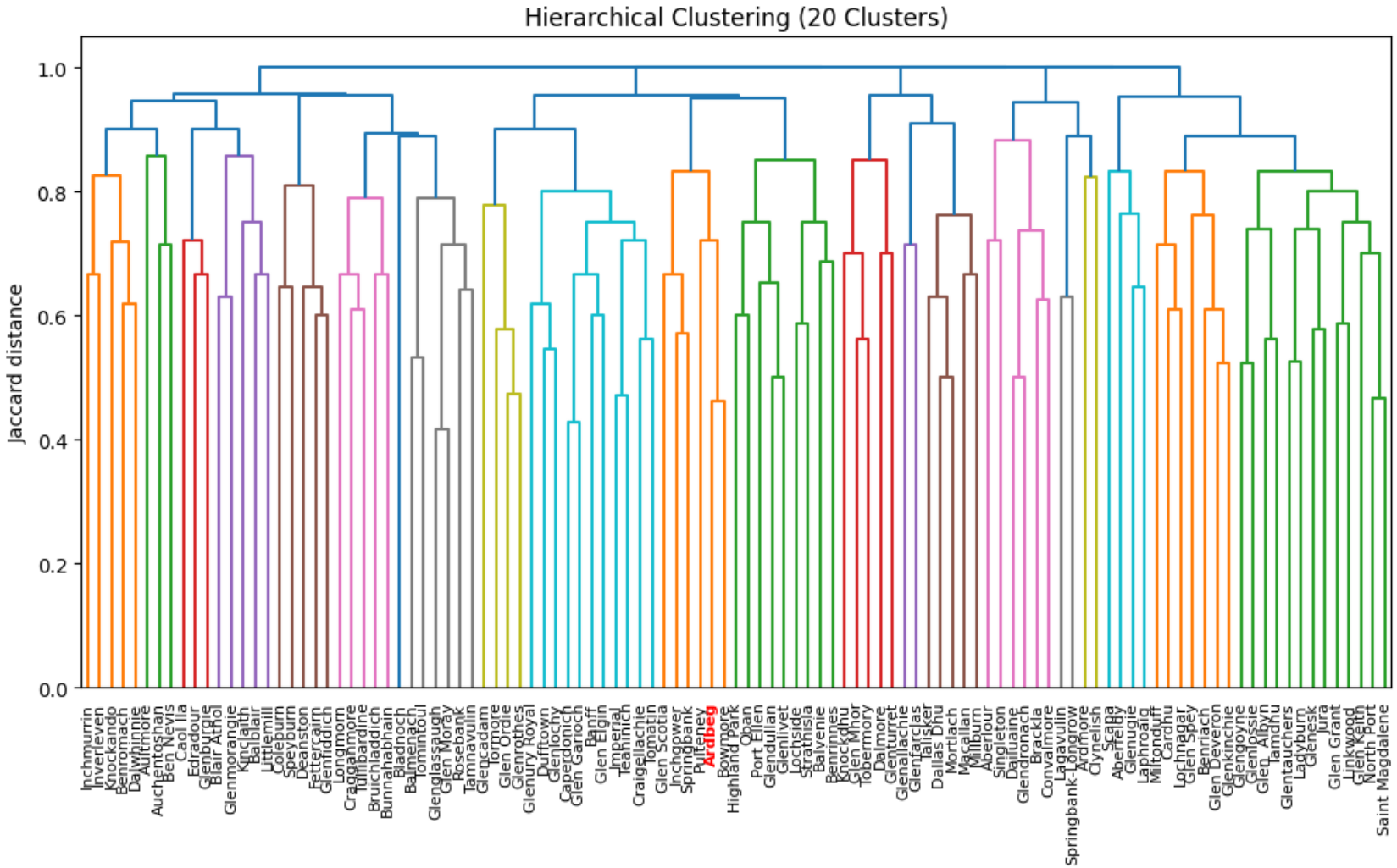
Dendrogram for whisky example

Built by merging closest “groups” to each other at each level of the hierarchy

Lots of variants (depending on how group-closeness is measured)

Used “complete linkage” here: Merge groups whose farthest elements are closest

20 Clusters



Ardbeg's cluster mates: Ardbeg, Bowmore, Glen Scotia, Inchgower, Pulteney, Springbank

Features of Ardbeg's cluster:

nose.sea: 1.000
fin.salt: 0.833
body.light: 0.833
body.med: 0.833
pal.sweet: 0.833

(Top 5 expressed tasting notes)

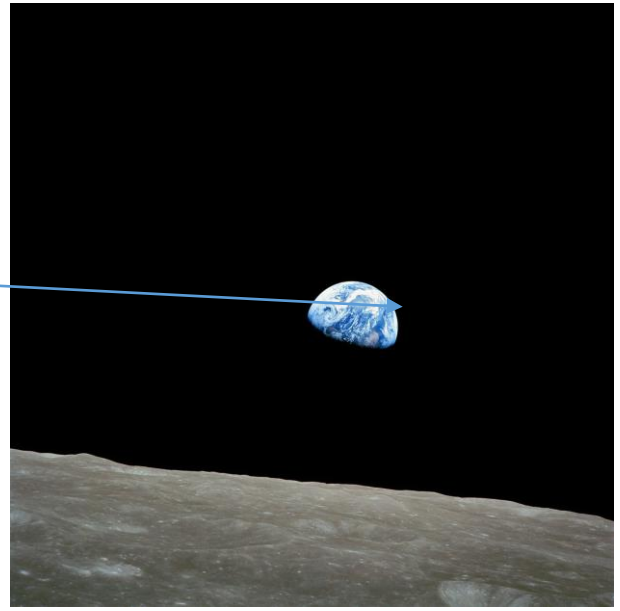
3. Introduction to Encoding: Dimension Reduction and PCA

Encoding

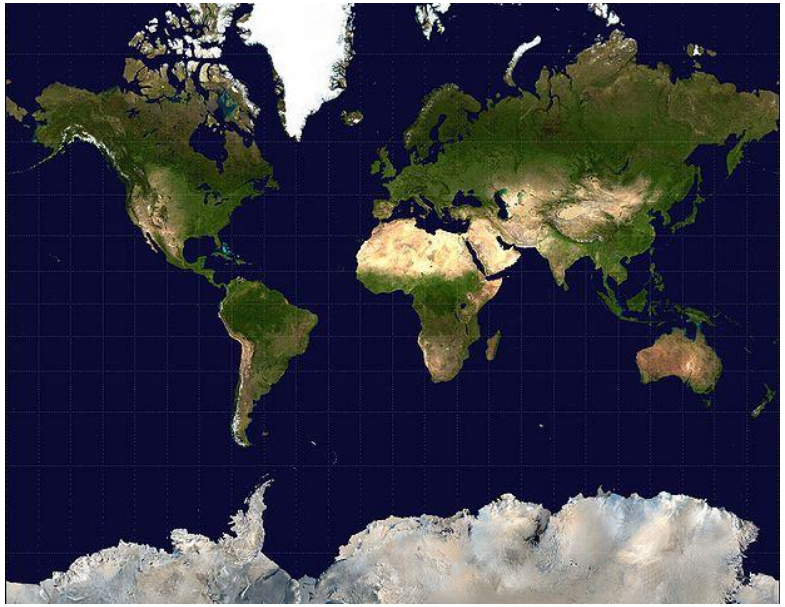
Encoding is jargon for taking input data and producing a representation (aka embedding) that summarizes the input

- Usually (but not always) reduces dimension relative to input data
- Goal to produce representation that captures relevant features for some task

Data: 3-dimensional



Encoder



Representation: 2-dimensional

Linear Dimension Reduction

Input: Observed variables $(X_1, X_2, \ldots, X_p)$ where p may be large

Output: Constructed variables $(F_1, F_2, \ldots, F_K)$ where K is small and

$$F_j = \beta_{j,1} X_1 + \cdots + \beta_{j,p} X_p$$

Constructed variables are linear combinations of input variables

- Lots of ways this could be done – need a target (next slide)
- Number of constructed variables K is a tuning parameter
- Typically, variables are standardized before applying linear dimension reduction

Constructed variables might be termed factors/embeddings/latent variables/representations/encodings/…

- i.e. lots of different jargon depending on field and application

Principal Components Analysis (PCA)

Most common linear dimension reduction procedure is PCA

PCA goal: Find embeddings $F_j = \beta_{j,1}X_1 + \cdots + \beta_{j,p}X_p$ such that

1. Coefficients satisfy $\sum_{v=1}^{p} \beta_{j,v}^2 = 1$
2. Embeddings have maximal variance
3. Embeddings are uncorrelated

Mathematical statement:

$$(\beta_{j,1}, \ldots, \beta_{j,p}) = \arg\max \mathrm{Var}(F_j)$$

subject to

$$\sum_{v=1}^{p} \beta_{j,v}^2 = 1$$

$\mathrm{Corr}(F_j, F_{j-m}) = 0$ for all $m \in \{1, \ldots, j-1\}$

Verbal description:

The j[th] principal component is

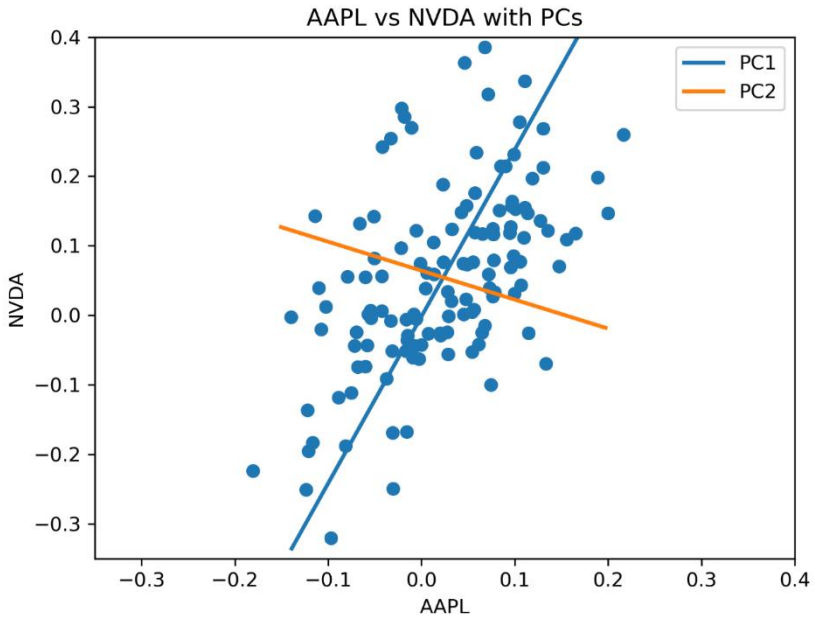
1. the direction
2. that captures as much variance as possible
3. while being uncorrelated with all previous principal components

Code for PCA finance illustration

PCA Illustration

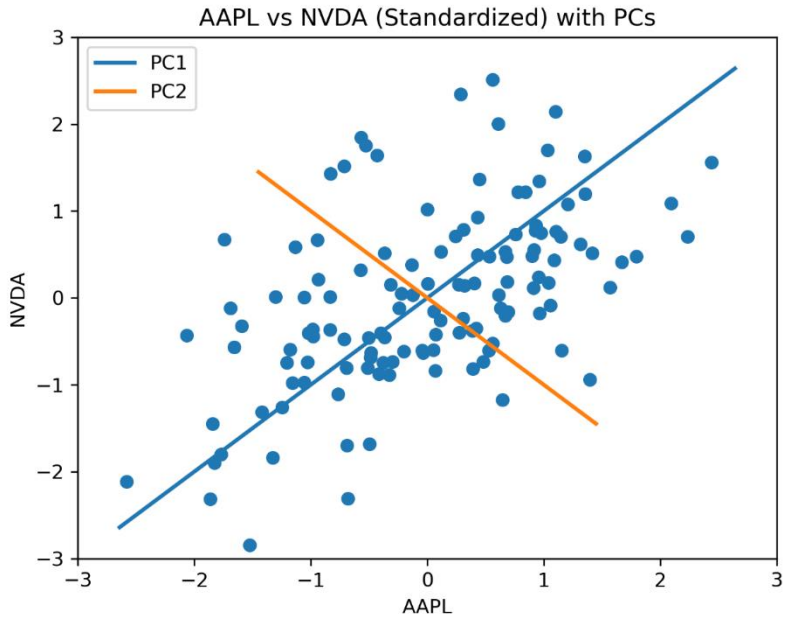
PCA in monthly asset returns (AAPL vs NVDA):

PCs in raw returns



	Direction AAPL	Direction NVDA	Fraction Explained
PC1	0.384	0.923	0.834
PC2	0.923	-0.384	0.166

PCs in standardized returns



	Direction AAPL	Direction NVDA	Fraction Explained
PC1	0.707	0.707	0.769
PC2	0.707	-0.707	0.231

Code for PCA macro illustration

Macro Example

Typically, interested in PCA in settings with many variables

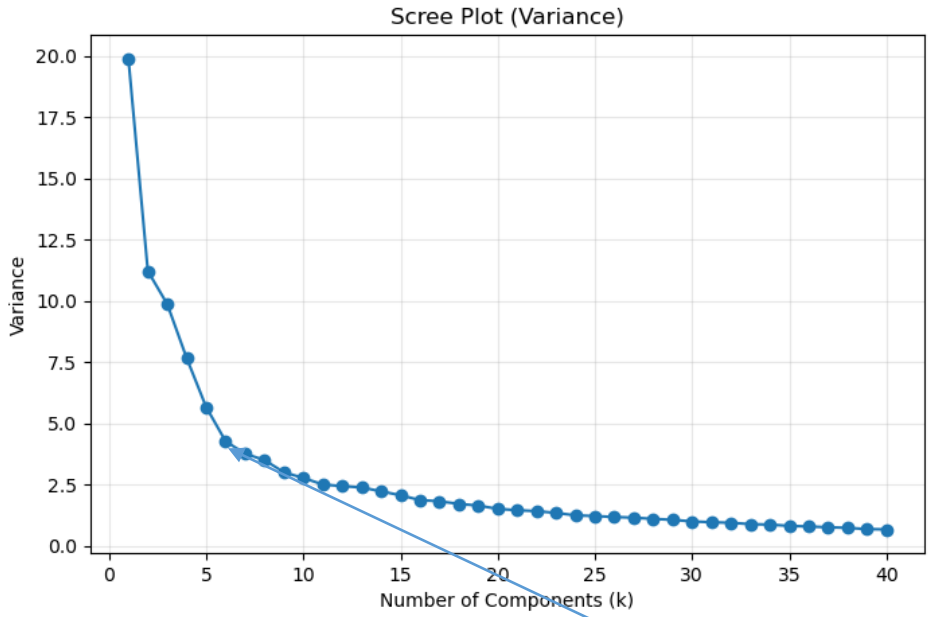
Many time series capturing different aspects of the macroeconomy

- Many seem “similar”
  - E.g. CPI: Apparel, CPI: Transportation, CPI: Medical Care, …
  - Do we really need all of them?
- Maybe we want a few features that “summarize” the state of the macroeconomy

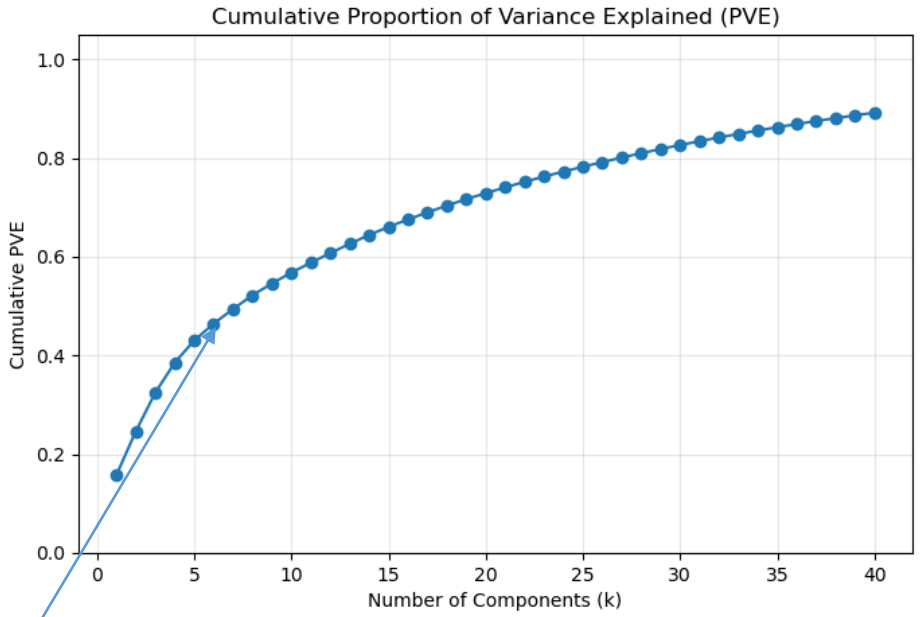
Let’s look at data from the Federal Reserve system

- Monthly data, 396 months, 126 macro time series
  - Transformed according to recommended practices
    - e.g., McCracken, M.W., Ng, S., 2015; FRED-MD: A Monthly Database for Macroeconomic Research, Federal Reserve Bank of St. Louis Working Paper 2015-012. URL https://doi.org/10.20955/wp.2015.012)
  - Standardized

Choice of Number of Factors/Components



Variance of each component



Total Fraction of Variation Explained

Seems like drop off after 6 components.

Explain around 50% of the variation.

If we had a clear downstream target, we could try to choose on the basis of that target.

Loadings: Interpreting Components

Loadings are commonly employed to help interpret components.

- Loadings are directions (β's) scaled by component standard deviations
- Help understand what X's are important contributors to the factors

Top 10 variables: PC1

Variable Name	|Loading|
All Employees: Total nonfarm FD log	0.866
IP: Manufacturing (SIC) FD log	0.848
IP: Final Products and Nonindustrial Supplies ...	0.829
All Employees: Service-Providing Industries F...	0.827
IP Index FD log	0.813
Capacity Utilization: Manufacturing FD	0.801
IP: Durable Materials FD log	0.787
All Employees: Trade, Transportation & Utilit...	0.779
IP: Final Products (Market Group) FD log	0.777
All Employees: Goods-Producing Industries FD log	0.756

Employment and Production

Top 10 variables: PC2

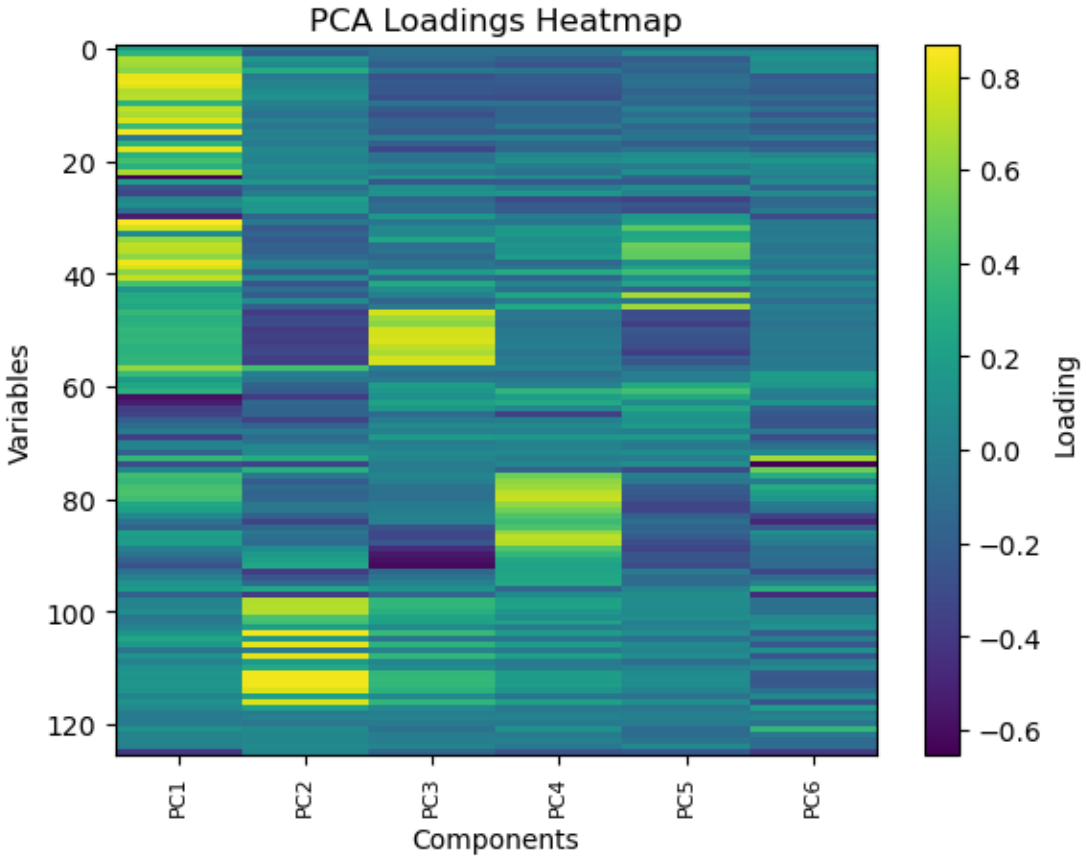
Variable Name	|Loading|
CPI : All Items SD log	0.835
CPI : All items less medical care SD log	0.834
CPI : All Items Less Food SD log	0.833
CPI: All Items Less Shelter SD log	0.832
CPI : Transportation SD log	0.809
CPI : Commodities SD log	0.805
Personal Cons. Expend.: Chain Index SD log	0.783
Personal Cons. Exp: Nondurable goods SD log	0.779
PPI: Processed goods for intermediate demand S...	0.697
PPI: Personal consumption goods SD log	0.689

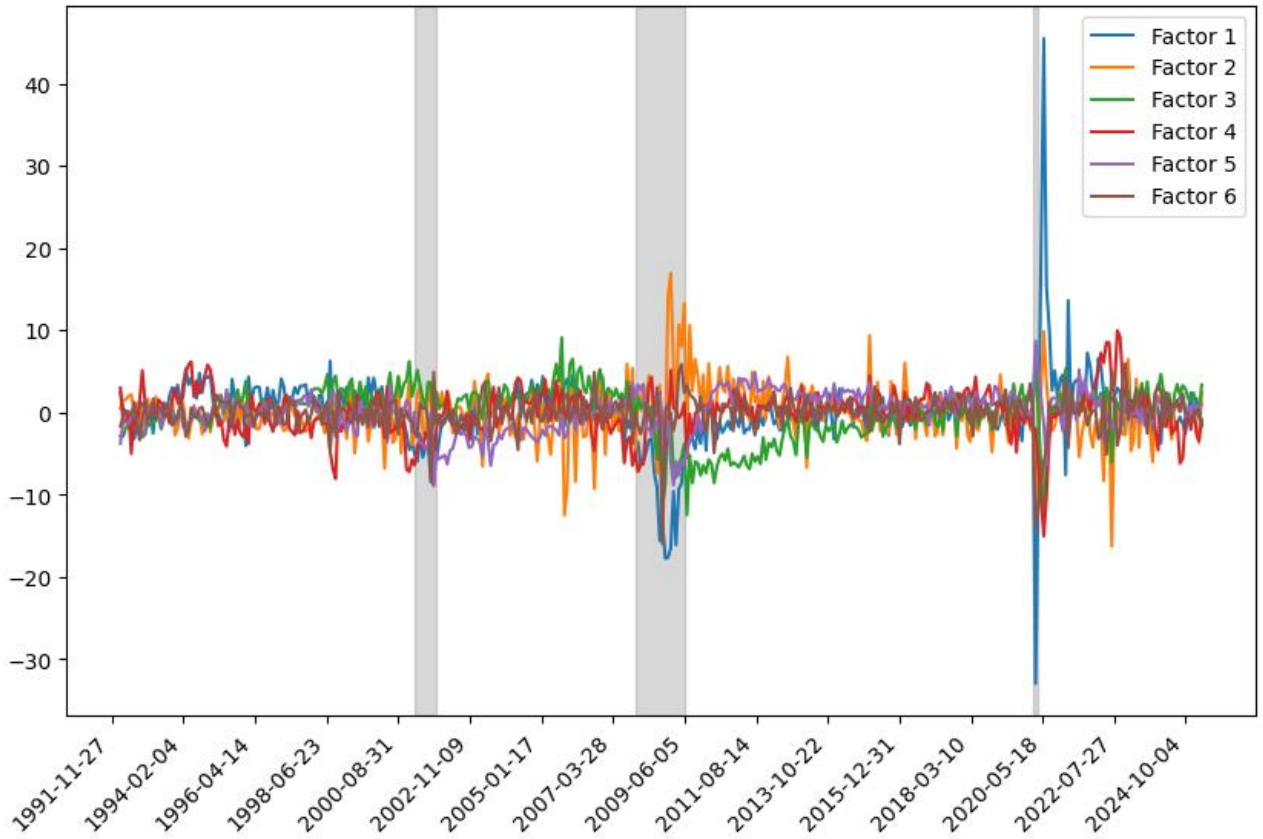
Inflation

Visualizing the Components

Time series plot of the extracted factors

What can we do with them?





Decoding

If our principal components, $F_1, \ldots, F_k$ effectively encode the information in the original features $X_1, \ldots, X_p$, it seems like we should be able to decode from $F_1, \ldots, F_k$ to get back a guess for $X_1, \ldots, X_p$

- A decoder takes an extracted representation and tries to reconstruct the original space (or some other target)

Given that PCA is a linear encoder, it is natural to consider a linear decoder:

$$\hat{X}_j = \lambda_{j,1} F_1 + \cdots \lambda_{j,K} F_K$$

- $\hat{X}_j$ is our decoded estimate of X_j

Many application of this idea. Let's look at matrix completion

- Combination of supervised and unsupervised learning
- An approach to fill in missing data

Netflix Problem

The "Netflix problem" is an example of matrix completion.

- Given a partial dataset of ratings, want to fill in/complete the data.
- Basic idea:
  - Customer i's rating for movie j can be represented as $X_{i,j} = \lambda_{i,1}F_{j,1} + \cdots \lambda_{i,K}F_{j,K} + \text{noise}$
    - $F_{j,1}, \ldots, F_{j,K}$ represent the (latent) characteristics of movie j
      - Estimated by the encodings from PCA
    - $\lambda_{i,1}, \ldots, \lambda_{i,K}$ represent customer i's tastes for movie characteristics
      - Estimated by the decoder trying to reconstruct ratings from principal components

In real life:

Make some adaptations to use all the available data (not just complete cases).

Would use a nonlinear encoder/decoder structure for something like the actual Netflix problem.

	Terminator II	2001: A Space Odyssey	Bambi	Gladiator
Customer 1	★★★★	★★	??	★★★★★
Customer 2	★★★	??	★★★★★	??
Customer 3	★	★★★★★	??	??
Customer 4	★★★★	★★	★★★★	★★★★
Customer 5	??	★★	★★★★★	★★★★

Code for IPCA macro illustration

Filling in the Missing Macro Data

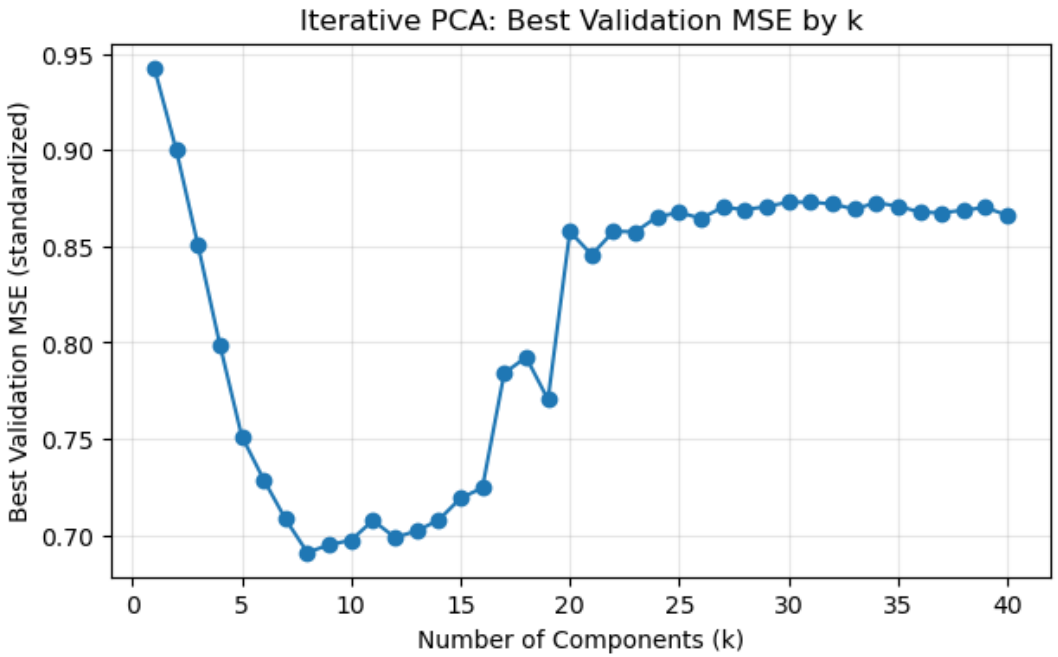
The macro data have 1002 missing entries across all variables

- 1% of the total entries
- Using only complete observations uses only 396 of the available 796 observations
  - I.e., we drop more than half the observations if we use only dates that have complete information on all 126 of the macro series

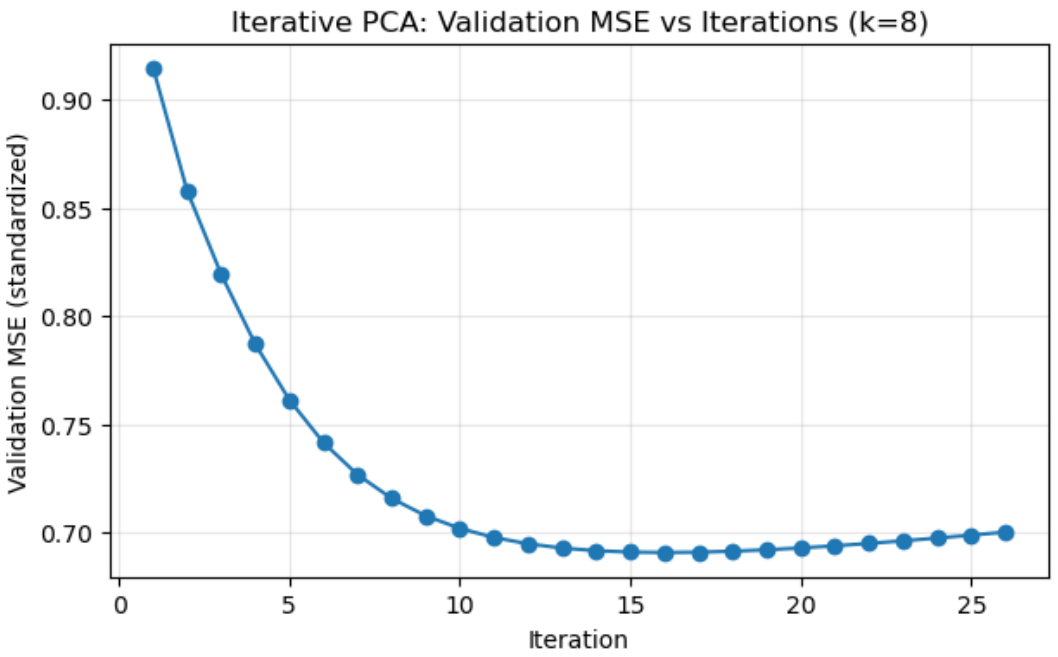
We can use the idea of PCA to use (almost) all the data *and* fill in the missing values

- Split to training and testing data by masking some of the observed data
- Use the training data to estimate the encoder (including using different K) and decoder
  - Not quite using vanilla PCA – doing "iterative" PCA to allow us to use all data
- Evaluate on the masked data to choose tuning parameters
- Use the best encoder/decoder structure to fill in the missing values

Tuning



$K = 8$ gives best validation performance for predicting masked observations



We have to start from guesses for the missing (masked) values and update them iteratively to be able to use (almost) all the data

Completed Series

Some look “better” than others

No way to really know – we don’t have the real data

Other ways to fill in missing data.

Basic notions of encoder/decoder structure and masking for “self-supervised” learning important in modern applications